Aula 2 - Métodos e Comportamentos



🚀 Aula Prática

Projeto Incremental: Sistema de Mobilidade Urbana (FiapRide)

Módulo: Programação Orientada a Objetos Aplicada

Bem-vindos de volta, equipe! Na nossa última aula, nós demos o primeiro passo criando a estrutura e os atributos do nosso sistema com a classe `Passageiro`. Nós definimos *o que o objeto é* e *o que ele tem*. Hoje, vamos dar vida a ele definindo *o que ele faz*.

🎯 Objetivo de Aprendizagem

Ao final desta prática, você será capaz de compreender os métodos como ações que alteram ou consultam o estado de um objeto, e aplicar a criação de funções curtas para resolver regras de negócio reais.

📖 Teoria: O que são Métodos e Comportamentos?

Até agora, nossos objetos são como fotografias: eles guardam informações (estado), mas estão "parados". Na aula passada, vimos os **Atributos**, que representam as características do nosso objeto. Por exemplo, o passageiro *tem* um `nome` e *tem* um `saldo`.

Mas no mundo real, as coisas agem e reagem.

Na Programação Orientada a Objetos, nós chamamos as ações de um objeto de **Métodos** (ou comportamentos).

- **Se os Atributos são os *substantivos* e *adjetivos* do nosso sistema, os Métodos são os *verbos*.**

Para que eles servem?

1. **Alterar o estado do objeto:** Um método pode modificar os atributos. (ex: Quando o passageiro recarrega o app, o método altera o atributo `saldo`).
2. **Comunicar-se com outros objetos:** Um objeto pode "chamar" o método de outro. (ex: O objeto `Motorista` pode chamar o método de pagar do `Passageiro`).
3. **Proteger regras de negócio:** Impedir que ações inválidas aconteçam (ex: sacar dinheiro sem ter saldo).

Sem métodos, nossos objetos seriam apenas listas de dados sem inteligência. Os métodos são os motores que fazem o software rodar!

🐙 Preparando o Terreno: Baixando seu Projeto do Git

Antes de começarmos a programar, precisamos garantir que você está com a versão mais recente do projeto que sua equipe iniciou na aula passada.

Passo a passo para baixar/atualizar seu repositório:

1. **Abra o Terminal** (ou Git Bash, se estiver no Windows).
2. **Navegue até a pasta** onde você guarda seus projetos usando o comando `cd` (ex: `cd documentos/faculdade/poo`).
3. **Se você ainda não tem o projeto na sua máquina:** Faça o clone do repositório da sua equipe copiando a URL lá do GitHub/GitLab

```
git clone https://github.com/usuario/nome-do-projeto.git
```

1. **Se você já tem a pasta do projeto da aula passada:**Entre na pasta do projeto e baixe as atualizações que sua dupla possa ter feito:Bash

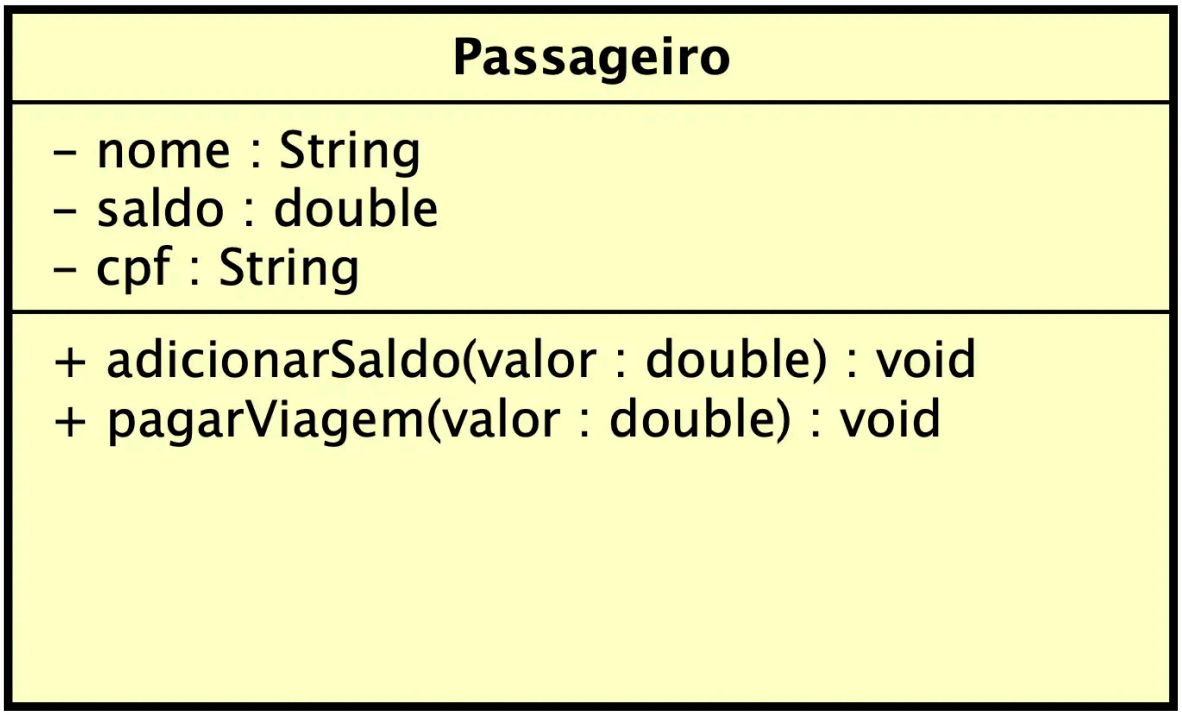
```
cd nome-do-projeto
git pull origin main
```

(Lembre-se de substituir `main` por `master` caso a sua branch principal tenha esse nome).

🔍 Exemplo Guiado: O Objeto "Passageiro"

Para ilustrar, vamos olhar para o nosso aplicativo de mobilidade (FiapRide). Na aula passada, nosso diagrama UML do `Passageiro` só tinha a primeira e a segunda divisão (Nome da Classe e Atributos).

Agora, observe o **incremento no UML**: adicionamos a terceira divisão, que representa os **Métodos** (`adicionarSaldo` e `pagarViagem`).



💻 Prática (Mão na Massa)

Nossa startup de mobilidade está implementando a carteira digital dos usuários. Atualmente, o objeto `Passageiro` que você criou na aula passada está estático. Precisamos que ele consiga adicionar fundos para pagar por suas corridas.

A Missão:

1. Abra o projeto da aula anterior.
2. Adicione o comportamento de recarga à classe `Passageiro` criando o método `adicionarSaldo(valor)`.
3. Adicione o comportamento de uso criando o método `pagarViagem(custo)`.
4. **Regra de Negócio Crítica:** Um passageiro não pode adicionar um saldo negativo (não faz sentido depositar "-20 reais") e não pode realizar uma viagem se não tiver saldo suficiente.

🧉 Exemplo de Estrutura Inicial (Java)

> Arquivo Passageiro.Java

```java
public class Passageiro {
    public String nome;
    public String cpf; // Novo Atributo
    public double saldo;

    public Passageiro(String nome, String cpf) {
        this.nome = nome;
        this.cpf = cpf; // Novo Atributo
        this.saldo = 0.00;
```

```
    }

    public void adicionarSaldo(double valor) {
        // Regra de negócio: O valor da recarga deve ser positivo
        if (valor <= 0) {
            System.out.println("Erro: O valor de recarga deve ser maior que zero.");
break;
            return; // Interrompe a execução do método
        }
        this.saldo += valor;
        System.out.println("Recarga realizada. Novo saldo: " + this.saldo);
    }

    public void pagarViagem(double custo) {
        // Regra de negócio: O custo deve ser positivo e o saldo deve ser suficiente
        if (custo <= 0) {
            System.out.println("Erro: O custo da viagem é inválido.");
            return;
        }
        if (this.saldo < custo) {
            System.out.println("Erro: Saldo insuficiente para realizar a viagem.");
            return;
        }
        this.saldo -= custo;
        System.out.println("Viagem paga. Saldo restante: " + this.saldo);
    }
}
```

Arquivo SistemaPrincipal.Java

```
package br.com.fiapride.main;

// Importamos a classe Passageiro para que o sistema a reconheça
import br.com.fiapride.model.Passageiro;

public class SistemaPrincipal {

    public static void main(String[] args) {
        // INSTANCIAÇÃO
        // O comando 'new' aloca memória para um novo objeto.
        // Criando o primeiro passageiro (Objeto 1)
        Passageiro passageiro1 = new Passageiro("Ana Silva", "222");
        System.out.println("Regarga passageiro 1");
        passageiro1.adicionarSaldo(50.0);

        // Criando o segundo passageiro (Objeto 2)
        Passageiro passageiro2 = new Passageiro("Carlos Souza", "333");
        System.out.println("Regarga passageiro 2");
        passageiro2.adicionarSaldo(12.5);
```

```
        System.out.println("--- Sistema FiapRide ---");
        System.out.println("Passageiro: " + passageiro1.nome + " | Saldo: R$ " + passageiro1.saldo +
" | CPF: " + passageiro1.cpf);
        System.out.println("Passageiro: " + passageiro2.nome + " | Saldo: R$ " + passageiro2.saldo +
" | CPF: " + passageiro2.cpf);

        System.out.println("Pagando viagem do passageiro 1");
        passageiro1.pagarViagem(20);
        System.out.println("Pagando viagem do passageiro 2");
        passageiro2.pagarViagem(20);

    }
}
```

Exemplo de Estrutura Inicial (Python)

```python
class Passageiro:
    def __init__(self, nome, cpf):
        self.nome = nome
        self.cpf = cpf # Novo Atributo
        self.saldo = 0.0

    def adicionar_saldo(self, valor):
        # Regra de negócio: O valor da recarga deve ser positivo
        if valor <= 0:
            print("Erro: O valor de recarga deve ser maior que zero.")
            return # Interrompe a execução do método

        self.saldo += valor
        print(f"Recarga de R${valor:.2f} realizada. Novo saldo: R${self.saldo:.2f}")

    def pagar_viagem(self, custo):
        # Regra de negócio: O custo deve ser positivo e o saldo deve ser suficiente
        if custo <= 0:
            print("Erro: O custo da viagem é inválido.")
            return

        if self.saldo < custo:
            print("Erro: Saldo insuficiente para realizar a viagem.")
            return

        self.saldo -= custo
        print(f"Viagem paga: R${custo:.2f}. Saldo restante: R${self.saldo:.2f}")
```

Boas Práticas: O arquivo `.gitignore`

Quando compilamos nosso código Java ou Python, o computador gera arquivos auxiliares (como os `.class`) e as nossas IDEs (Eclipse, IntelliJ, VS Code) criam pastas de configuração (como `.idea` ou `.vscode`).

Nós nunca devemos enviar esses arquivos para o GitHub! Eles deixam o repositório pesado, causam conflitos entre os membros da equipe e expõem configurações locais da sua máquina.

Para resolver isso, usamos um arquivo chamado `.gitignore`. Ele funciona como uma "lista de bloqueio": tudo o que estiver escrito nele, o Git vai ignorar completamente.

Exemplo prático para Projetos em Java: Crie um arquivo chamado exatamente `.gitignore` na raiz do seu projeto e adicione o seguinte conteúdo:

```
# Ignorar arquivos compilados do Java
*.class
# Ignorar pastas geradas por IDEs (VS Code, IntelliJ, Eclipse)
.vscode/
.idea/
*.iml
.metadata/
# Ignorar pastas de build e dependências locais
bin/
target/
```

Com isso, o seu repositório ficará limpo, contendo apenas o que importa: **o seu código-fonte**.

🧠 Pergunta de Reflexão

Reflita:

Se nós podemos simplesmente fazer `passageiro.saldo = passageiro.saldo + 100` diretamente no código principal, por que dá tanto trabalho criar um método específico chamado `adicionarSaldo(valor)` para fazer isso? Quais seriam os riscos para a nossa startup de mobilidade se deixássemos qualquer programador alterar o saldo diretamente?

💻 Desafio Prático (Mão na Massa)

Agora é com você! Você vai evoluir o seu projeto do trabalhando em cima do objeto que você criou na aula passada.

A Missão:

1. **Atualize o UML:** Adicione pelo menos 2 (dois) métodos no diagrama de classe do seu projeto.
2. **Crie os Métodos:** No código, implemente esses métodos para que eles alterem o estado de pelo menos um atributo.
3. **Proteja o Estado:** Para cada método, inclua uma **regra de negócio** (validação com `if`). Não permita que o estado do seu objeto fique inconsistente.
4. **Teste:** Instancie o objeto e teste chamando os métodos com valores válidos e inválidos.
5. **Documente:** Crie (ou atualize) o arquivo `README.md` do seu projeto. Explique de forma clara o funcionamento da sua classe, o que o seu objeto representa no mundo real e como usar os métodos que você acabou de criar.
6. **Versionamento:** Faça o commit das atualizações no projeto criado na aula passada rodando os seguintes comandos no terminal:

```
git add .
git commit -m "feat: adiciona métodos de comportamento e atualiza README"
git push origin main
```

🖌️ Critérios de "Clean Code" (Código Limpo)

Para que sua entrega seja aprovada no nosso *Code Review*:

- **Ações são Verbos:** Os nomes dos seus métodos devem ser verbos no infinitivo (ex: `atualizarEstoque()`, `registrarAlta()`).
- **Funções Pequenas:** O método deve ter uma única responsabilidade. Valide a regra e altere o estado. Nada de criar métodos gigantes!
- **Nomes Significativos para Argumentos:** O parâmetro deve ter um nome claro (ex: `quantidadeComprada`, `novoStatus`), evite letras soltas (`q`, `x`, `val`).

Bons estudos!

